

Cloud, Docker, and Kubernetes Fundamentals

Deployment should make an application reproducible, observable, and recoverable. Containers package execution dependencies; orchestration coordinates instances and desired state. Neither automatically fixes an unsafe application, missing backups, incorrect secrets, or a database schema change incompatible with older versions.

For: Developers who want to package and deploy a service reliably and explain containers, orchestration, networking, configuration, and rollout without assuming every project needs Kubernetes.

Suggested pace: 4 weeks with local-first deployment | A working application required

What you will be able to do

- Build a reproducible container image and separate runtime configuration from code.
- Explain process isolation, image layers, networks, and persistent state.
- Distinguish readiness, liveness, startup, and graceful termination behavior.
- Choose a deployment model and rollback plan proportional to the workload.

Build the right mental model

A container is not a complete virtual machine

Containers share a host kernel under isolation and resource controls. An image captures filesystem and execution dependencies, while runtime configuration supplies environment-specific settings. Avoid embedding credentials and do not assume local writable container storage is durable.

Desired state is not application correctness

An orchestrator can restart a failed process, but a repeatedly failing application remains broken. Health probes should reflect meaningful readiness and liveness without causing cascading restarts when a dependency has a temporary problem.

Deployment includes data and compatibility

Rolling versions may run simultaneously. APIs, schema changes, and jobs must tolerate that overlap. A rollback of code does not necessarily reverse a destructive migration, so prefer staged compatible changes and a tested recovery strategy.

Study with evidence.

Start locally with a small service and container runtime. Introduce a cluster only after the deployment problem justifies it. Keep cloud spending bounded, use least privilege, and remove only resources created for the exercise.

Week 1: Package the application

Create a minimal image with explicit dependencies and a non-root runtime where appropriate. Separate build and runtime needs, document ports, and pass configuration through a safe mechanism. Test the image from a clean environment.

Deliverable and check: A reproducible image and a local run command that does not contain secret values in the repository.

Week 2: Runtime behavior

Add structured logs, resource limits, graceful signal handling, and persistent storage where required. Test startup failure, dependency timeout, and process termination. Distinguish application state from ephemeral container files.

Deliverable and check: A container that starts predictably, stops safely, and reports actionable health information.

Week 3: Orchestration concepts

Study Pods, Deployments, Services, configuration, secrets, probes, and resource requests/limits using a local or approved test cluster. Explain how traffic reaches a ready instance and what happens during rescheduling.

Deliverable and check: A small manifest set with an explicit service path and no claim that a replica count alone ensures availability.

Week 4: Rollout and recovery

Practice a rolling update and a rollback with a compatible schema. Observe readiness and termination during traffic. Compare managed application hosting with cluster operation and choose the simpler model that meets the need.

Deliverable and check: A runbook for deployment, validation, rollback, and cost/resource cleanup.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Deploy without dropping in-flight work

A service handles requests while a new version rolls out. Terminating an old instance immediately can interrupt accepted work, while leaving it ready during shutdown can attract new work indefinitely.

A worked approach

1. On shutdown, stop accepting new work or mark the instance unready according to the platform contract. Allow routing changes to propagate and define a bounded grace period for in-flight requests.
2. Drain existing requests, stop background consumers safely, and close resources only after they are no longer used. Requests that cannot finish within the deadline need a documented retry or failure policy.
3. Run a local load test during a rolling update and record errors, latency, and version overlap. Test a slow request and a dependency failure, not just fast happy-path traffic. Verify that probes do not restart healthy-but-waiting instances unnecessarily.

Expected result

The rollout has a controlled transition between versions and a bounded shutdown contract. Orchestration supports the behavior, but application-level draining and idempotent retry rules remain essential.

Verify the result

- Does the instance stop receiving new work before resources are destroyed?
- Are background jobs and HTTP requests both considered?
- Can the older version still operate with the deployed schema?

Extend the exercise

Add a database column through an expand-and-contract migration. Demonstrate old and new application versions coexisting before removing the old field in a later release.

Make the learning stick.

Should every project use Kubernetes?

No. It adds operational and conceptual cost. A managed application platform or simpler deployment may meet the workload's needs with less maintenance. Choose orchestration for requirements, not as a portfolio decoration.

What is the difference between readiness and liveness?

Readiness controls whether an instance should receive traffic; liveness indicates whether restarting may recover a stuck process. Incorrect probes can route traffic too early or create restart storms during dependency outages.

Are secrets safe because they are environment variables?

Not automatically. They can appear in process inspection, logs, crash reports, or configuration exports. Use the platform's secret-management controls, least privilege, rotation, and careful logging rather than relying on one representation.

Avoid these traps

- Storing important data only in an ephemeral container filesystem.
- Using liveness probes that restart every instance during a dependency outage.
- Rolling back code after an incompatible destructive schema migration.

Ready for the next step?

- Images and runtime configuration are reproducible and separated.
- Health and shutdown contracts are tested.
- Persistent data and secrets have explicit ownership.
- Deployment and rollback include version compatibility.

Reliable next reads

- [Docker getting started](#)
- [Kubernetes concepts](#)
- [Kubernetes probes](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.